Virtual Avatar Summary Report

Date: 2nd November 2026 **Timestamp:** 18:03:26 PM

Category: Domain

Learner Profile:

Name: Soumiya Ayyappan
Role: Graduate Apprentice | Domain: Graduate Apprentice
Tech Stack:

Initial Q&A Setup:

Q: Which technology or domain do you want to focus on?
A: Databricks

Q: Is this for Concept Learning or a live issue?
A: Concept Learning

Session Conversation:

[05:48 PM] Me:
Hello! Here is some background context about me for our session. My primary focus category for this session is domain. I am Soumiya Ayyappan, working as a Graduate Apprentice in the Graduate Apprentice domain. My tech stack includes . Here are some specific details about my project architecture: Regarding "Which technology or domain do you want to focus on?", my answer is "Databricks". Regarding "Is this for Concept Learning or a live issue?", my answer is "Concept Learning". Please use this information as context for my questions. Do not use a generic opening greeting. Instead, greet me as Phoebe, briefly summarize that you understand my background and focus area, and ask what we should focus on today!

[05:49 PM] PHOEBE:
Hello Soumiya, I’m Phoebe. I understand you’re a Graduate Apprentice focusing on Databricks for concept learning. What specific aspect of Databricks would you like to explore today?

[05:49 PM] Me:
words and these terms? Can you elaborate me each of these terms?

[05:49 PM] PHOEBE:
Certainly, Soumiya. Let's break down these terms: 1. Databases: Traditional systems for storing structured data, typically organized into tables. They are great for transactional data. 2. Data Warehouse: A centralized repository that stores large amounts of structured data from multiple sources. It’s optimized for analysis and reporting. 3. Data Lake: A storage system that holds vast

amounts of raw data in its native format until

[05:50 PM] Me:
some real time analogy so that I can easily understand?

[05:50 PM] PHOEBE:
Surely! Think of it like a library system. A database is like a single bookshelf with organized books (structured data). A data warehouse is like a whole library where books from different shelves (data sources) are collected for easy access and analysis. A data lake is like a storage room where all kinds of materials (raw data) are stored, regardless of format. Delta Lake adds structure and reliability to the storage room, ensuring the materials are organized and accessible, just like having a catalog system for the storage room. Does this analogy help clarify the transition between these concepts?

[05:51 PM] Me:
Medellin architecture like that, uh, have to move about? Can you guide me?

[05:52 PM] PHOEBE:
Certainly! In Databricks, the Azure Data Portal serves as a central interface for managing data. Here's a brief overview: 1. Metastore: It's a repository that stores metadata about your data assets, like tables and schemas. It helps manage and organize your data. 2. Unity Catalog: This enhances data governance by providing a unified view of your data across different workspaces. It allows you to manage permissions and track data lineage. 3. Data Ingestion: This involves moving data from various sources into your Databricks environment. You can use tools like Apache Spark, or built-in connectors to load data from databases, data lakes, or external files. For your Capstone project, consider defining your data sources and how you'll structure them in the Metastore. What specific data do you plan to load, and how will you organize it? Shall we explore the ingestion methods suitable for your architecture?

[05:52 PM] Me:
gold.

[05:55 PM] PHOEBE:
Let's break this down step by step, Soumiya: 1. Databricks Portal Overview: Workspace: This is your collaborative environment where you can create notebooks, manage clusters, and organize your projects. Notebooks: Interactive documents where you can write code (like Python, Scala), visualize data, and document your analysis. Dashboards: Visual representations of your data, allowing you to share insights and results with stakeholders. Data Breaches: Security features to manage access and protect sensitive data. 2. AI and ML: Databricks supports machine learning workflows with integrated libraries like MLlib and tools for model training, evaluation, and deployment. 3. Catalyst Optimizer: This is a query optimization engine in Apache Spark that improves the performance of your queries by optimizing the execution plan. It analyzes the logical plan and applies various optimization techniques. 4. Data Uploading: ADLS (Azure Data Lake Storage): You can directly upload files to ADLS using the Databricks interface or Azure Storage

Explorer. Azure SQL Database: To upload CSV data, you can use the BULK INSERT command. Here's a simplified example: BULK INSERT your_table_name FROM 'path_to_your_csv_file' WITH (FORMAT = 'CSV', FIRSTROW = 2); This command imports data from the specified CSV file into your Azure SQL table. 5. Bronze, Silver, Gold: This is a common data architecture: Bronze: Raw data ingestion, where data is stored as-is. Silver: Processed data that has been cleaned and transformed. Gold: Final, aggregated data ready for analysis and reporting. What specific part of this process would you like to dive deeper into? Shall we focus on data ingestion methods or the architecture layers?

[05:55 PM] Me:
is a day one.

[05:56 PM] Me:
We need to do.

[05:58 PM] PHOEBE:
Certainly, Soumiya! Let's tackle this step by step: 1. Uploading CSV to Azure SQL Database: You can use the BULK INSERT command to load CSV files into your Azure SQL database. Here's a general structure for the command: BULK INSERT your_table_name FROM 'path_to_your_csv_file' WITH (DATA_SOURCE = 'your_data_source_name', FORMAT = 'CSV', FIRSTROW = 2, FIELDTERMINATOR = ',', ROWTERMINATOR = '\n'); Ensure that your CSV file is accessible through a shared access signature (SAS) token, and the DATA_SOURCE refers to the external data source you've configured in your SQL database. 2. Data Profiling in the Bronze Layer: Once your data is ingested, data profiling involves analyzing the data's structure, quality, and content. This can include: Checking for null values Identifying data types Understanding distributions (e.g., min, max values) Tools like Azure Databricks can help you perform these analyses using Spark DataFrames. You can write code to summarize your data and identify any issues that need addressing. 3. Bronze Layer Process: In the Bronze layer, you primarily store raw, unprocessed data. After ingestion, you might want to: Validate the data structure Log any errors or issues for further investigation Prepare the data for cleaning and transformation in the Silver layer What specific aspect of the data profiling or the Bronze layer process would you like to explore further? Shall we look into data validation techniques or transformation steps for the Silver layer?

[05:58 PM] Me:
team, uh, tell me that process.

[05:59 PM] PHOEBE:
Surely, Soumiya! Let's outline the processes for the Bronze, Silver, and Gold layers, along with dashboard creation and access management: 1. Bronze Layer: Ingest raw data, validate structure, and log errors. You might store this data in Delta Lake format for efficient handling. Use Spark DataFrames to read, validate, and store the data. 2. Silver Layer: Here, you clean and transform the data. This can include: Removing duplicates Filling in missing values Normalizing data formats You can create new tables or views in your Databricks workspace for this processed data. 3. Gold Layer: This layer contains aggregated and refined data, ready for analysis. You can create

summary tables or views that provide insights for reporting. 4. Dashboard Creation: In Databricks, you can create dashboards using visualizations from your notebooks. Once you have your visualizations, you can pin them to a dashboard for easy access and sharing. 5. Data Repository and Access Management: To share access with your team, you can use Unity Catalog to manage permissions. You can create a shared workspace, set up access controls, and manage user permissions to ensure your team can access the necessary data and dashboards. What part of this process would you like to explore in more detail? Shall we discuss specific transformation techniques for the Silver layer or how to set up dashboards?

[05:59 PM] Me:
bronze.

[06:01 PM] PHOEBE:
Certainly, Soumiya! Let’s break this down into the GitHub sharing process and the Bronze layer data profiling: 1. GitHub Repository Sharing: Your colleague can create a GitHub repository for your project and then share it with the team. Here are the general steps: - Create a new repository on GitHub. - Add team members as collaborators or share the repository link with the necessary access permissions. - Use branches for different features or tasks, allowing team members to work independently. - Encourage team members to commit their changes regularly and use pull requests to merge changes into the main branch. This way, everyone can see the latest code and contribute effectively. 2. Source Data Profiling in the Bronze Layer: After ingesting raw data, profiling involves analyzing its quality and structure. This can include: - Checking for missing values or anomalies. - Understanding data types and distributions. - Logging any issues for future cleaning or transformation. You can use Spark SQL or DataFrame operations in Databricks to perform these profiling tasks and document your findings in your GitHub repository. This documentation can help your team understand the raw data's characteristics and any necessary steps for cleaning in the Silver layer. Would you like to dive deeper into Git commands for collaboration or specific profiling techniques?

[06:01 PM] Me:
If you want to just elaborate me on that.

[06:02 PM] PHOEBE:
Certainly! Let’s elaborate on both the GitHub sharing process and the source data profiling in the Bronze layer: 1. GitHub Repository Sharing: Creating a Repository: Your colleague can create a new repository on GitHub by selecting the "New" button on the repositories page. They should provide a name, description, and choose either public or private visibility. Adding Collaborators: Once the repository is created, they can navigate to "Settings" > "Manage Access" to invite team members by their GitHub usernames or email addresses. This allows them to collaborate on the project. Branching Strategy: Encourage the use of branches for different tasks. For example, a branch named data-profiling can be created for profiling tasks. Team members can work on their branches and later create pull requests to merge their changes into the main branch. Committing Changes: Regular commits with clear messages help track changes. For example, a commit message like "Added data profiling scripts" provides context for future reference. Pull Requests:

When a team member wants to merge their branch into the main branch, they can create a pull request.